3. 系統設計與規格書 (Proposed Design & Specifications)

為解決大型語言模型在強化學習階段面臨的「極高運算資源需求」與「獎勵駭客 (Reward Hacking)」雙重挑戰，本專案針對 Kaggle: NVIDIA Nemotron Model Reasoning Challenge，提出一套基於「高斯群體強化適應 (Group Relative Policy Optimization, GRPO) 與參數高效微調 (LoRA)」的輕量化對齊架構。本節將詳細說明系統之軟硬體規格、核心模組設計與訓練管線流向。

3.1 軟硬體環境與限制規格 (System Environment & Constraints)

本專案立足於學術環境與個人競賽開發者的資源限制，系統架構將嚴格遵循以下硬體與軟體依賴規格：

- **硬體限制 (Hardware Constraint):** 單張消費級 / 工作站級 GPU，最高 VRAM 限制為 24 GB (如 NVIDIA RTX 3090 / 4090 或 A10G)。
- **基礎模型 (Base Model):** Llama-3-8B-Instruct (或 NVIDIA Nemotron-3-Nano-8B)，兼具優秀的開源基礎能力與適合單卡部署的參數體積。
- **核心框架 (Core Frameworks):** * 模型載入與量化： `Transformers` , `BitsAndBytes`
  - 高效微調模組： `PEFT` (Hugging Face)
  - 強化學習與推論加速： `TRL` (Transformer Reinforcement Learning), `vLLM` (用於 RL 探索階段的高速文本採樣)
- **驗證資料集 (Dataset):** NVIDIA Nemotron Challenge 提供之高質量 Reasoning Traces 與 Prompt 測試集。

3.2 核心模組設計 (Core Module Design)

本系統屏棄傳統 PPO 的四模型架構（Actor, Critic, Reference, Reward），改採「無評論家網路 (Critic-Free)」且「凍結主幹 (Frozen-Backbone)」的設計。

3.2.1 基礎模型與 4-bit 量化模組 (Base Model & Quantization)

- **規格：** 採用 NF4 (NormalFloat 4-bit) 量化技術載入 8B 基礎模型。
- **目的：** 將 8B 模型原本約需 16GB 的權重顯存佔用，大幅壓縮至 5~6 GB，為後續的 RL 生成採樣與梯度計算保留充足的 VRAM 空間。
- **設定：** 開啟 `double_quantization` 以進一步降低顯存，並設定 `bnb_4bit_compute_dtype=torch.bfloat16` 以維持訓練穩定性。

3.2.2 參數高效微調模組 (PEFT - LoRA Adapters)

為了讓模型具備學習新推理策略的能力，同時避免全參數微調的崩潰，本系統將低秩矩陣 (LoRA) 掛載於注意力機制與前饋網路中。

- **Target Modules (注入層):** `q_proj` , `k_proj` , `v_proj` , `o_proj` , `gate_proj` , `up_proj` , `down_proj` 。覆蓋 MLP 層是關鍵，因為最新研究指出模型的邏輯與知識重構大多發生於 MLP 層。

- **超參數規格:**
  - `Rank (r)`: 16 (確保足夠的特徵表達能力)
  - `Alpha (α)`: 32 (放縮係數)
  - `Dropout`: 0.05
- **可訓練參數量:** 預計小於總參數的 0.3% (~20M 參數)，極大化梯度更新效率。

3.2.3 可驗證獎勵環境建構 (Verifiable Rule-based Reward, RLVR)

捨棄傳統使用另一個 LLM 作為 Reward Model 的作法，徹底根絕獎勵駭客 (Reward Hacking)。針對數學與邏輯推理，我們開發一套 Python 規則驗證器作為環境回饋。

- **格式獎勵 (Format Reward):** 檢查輸出是否包含完整的 `<think> ... </think>` 思考軌跡標籤，以及最終的 `<answer> ... </answer>` 。格式正確給予 $R_{format} = 0.1$，缺失則給予負懲罰。
- **準確度獎勵 (Accuracy Reward):** 提取 `<answer>` 中的內容，透過字串正規化或 Python `eval()` (若為代數題) 與 Ground Truth 進行嚴格比對。正確給予 $R_{acc} = 1.0$，錯誤給予 $R_{acc} = 0.0$。
- **總獎勵函數:** $R_{total} = R_{format} + R_{acc}$

3.2.4 GRPO 強化學習優化器 (Group Relative Policy Optimization)

本專案的演算法核心。GRPO 透過對同一個問題採樣多個輸出，在群體內部計算優勢函數，完全消除了對 Critic Network 的依賴。

- **群組大小 (Group Size, G):** 設定為 $G = 4$ 到 8 (視 VRAM 剩餘空間而定)。
- **優勢函數估計 (Advantage Estimation):** 對於同一個 Prompt，模型生成 G 個不同的推論軌跡，計算各自的獎勵 R_i。將獎勵在群組內進行標準化： $A_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}$
- **KL 散度懲罰 (KL Penalty):** 計算當前 Policy (帶有更新後 LoRA) 與 Reference Policy (未更新前) 的 KL 散度，防止模型在追求高分的過程中失去語言的連貫性。懲罰係數 β 動態調整。

3.3 訓練管線與資料流 (Training Pipeline & Data Flow)

系統的執行將分為兩個連續的階段，構成完整的 Data Pipeline：

Phase 1: 監督式微調熱身 (SFT Warm-up)

- **輸入：** 競賽資料集中精選的 5,000 筆高質量、附帶完整思維鏈的問答對。
- **處理：** 利用標準的 Cross-Entropy Loss 對 LoRA 模組進行微調。
- **目標：** 讓基礎模型學會「先思考、再回答」的文本結構（學會使用 `<think>` 標籤），為後續 RL 提供良好的起始策略 (Base Policy)。
- **預期 Epochs:** 1 到 2 個 Epoch。

Phase 2: 強化學習自我演化 (RL Optimization via GRPO)

此為專案的核心執行迴圈，針對資料集中的 Prompt 進行無監督演化：

1. **環境給題 (Prompting):** 從訓練集抽取邏輯問題 x。
2. **多路採樣 (Generation):** 利用 vLLM 引擎，當前策略模型 (Base + LoRA) 對 x 生成 G 個不同的解答軌跡 $\{y_1, y_2, ...y_G\}$。設定 Temperature = 0.7 以確保探索多樣性。
3. **環境給分 (Scoring):** 將 G 個解答送入 RLVR 驗證器，獲得絕對獎勵 $\{R_1, R_2, ...R_G\}$。
4. **梯度更新 (Policy Update):** 計算標準化優勢函數 A_i，結合 KL 懲罰，計算 Policy Gradient Loss，並**僅對 LoRA 權重進行反向傳播 (Backpropagation)**。
5. **迭代：** 重複此過程，直到模型在驗證集上的 Average Reward 收斂。

3.4 預期技術突破與創新點 (Novelty & Technical Breakthroughs)

本設計架構的提出，預期能達成以下技術指標突破：

1. **運算資源的極限壓縮：** 傳統 8B 模型的 PPO RLHF 需要近 80GB VRAM；本架構透過 `4-bit Base + LoRA + GRPO` 的聯合設計，將峰值 VRAM 控制在 **18GB - 20GB 之間**，首次證明在單張 RTX 3090/4090 上執行完整 RL 推理對齊的可行性。
2. **根除模型崩潰與幻覺：** 透過放棄傳統 Reward Model，改採 Rule-based 可驗證獎勵，消除了神經網路裁判的偏見，迫使模型只能透過「真正提升邏輯嚴謹度」來獲取高分。
3. **湧現 (Emergent) 自我修正能力：** 預期在 GRPO 訓練中後期，模型會在 `<think>` 區塊內展現出「Wait, this is wrong, let me recalculate」的自我回溯與修正行為，這是單純 SFT 絕對無法達成的系統二 (System 2) 認知表現。